

Apunte de la materia, de la primera variable a la máquina de estados

C para software de vuelo

Ingeniería de Software de Vuelo para Sistemas Espaciales Críticos · 2.º cuatrimestre 2026

v0.1 — en construcción: módulos 1 y 2 de 12

Para qué es esto. Para aprender el C que usa la materia, en el orden en que lo da la cátedra, con cada programa **compilado de verdad** con los mismos flags que pide el práctico. Si un ejemplo de acá no compila, el apunte no se publica: lo chequea un script, no la buena voluntad.

Lo que no es. No resuelve los prácticos: los ejemplos están ambientados en el mismo OBC imaginario, pero con otros números y otros casos. Copiar y pegar no va a andar, y ésa es la idea.

El código sale en verde con su salida abajo, en negro, copiada de la corrida real en el gcc de Ubuntu: lo que ves impreso es lo que imprimió.

Los colores no son adorno: cada uno dice una cosa y sólo una.

Azul	en el texto, La idea : — lo que no hay que perder.
Rojo	en el texto, Ojo : — donde se traba casi todo el mundo.
Violeta	lo que pide la cátedra, sacado de sus clases y de sus TP.
Verde azulado	una mejora : algo que agregamos nosotros y no es de la cátedra. Se puede saltar.
Rosa	la posta: lo mismo, dicho en criollo y sin vueltas.

Índice

1 El programa mínimo	1
1.1 La plantilla de la cátedra	1
1.2 Compilar: de texto a programa	2
1.3 Leer un warning	2
1.4 Un programa de vuelo no termina	3
2 Variables y tipos	5
2.1 Qué es una variable	5
2.2 Los tipos de C, y por qué no alcanzan	5
2.3 La solución: tipos de ancho fijo	6
2.4 Con signo o sin signo: dos preguntas	6
2.5 Imprimir con el especificador correcto	7
2.6 La primera trampa: el desborde	7
2.7 La segunda trampa: la división entera	8

El programa mínimo

AL TERMINAR ESTE MÓDULO VAS A PODER

Escribir un programa en C con la estructura que pide la cátedra, compilarlo con los flags de la materia, leer un warning sin entrar en pánico, y explicar por qué un programa de un satélite no termina nunca.

C nació en 1972 en los laboratorios Bell, y medio siglo después sigue siendo el idioma en que se escribe casi todo el software que vuela. No es nostalgia. En un satélite el procesador es chico, la memoria es poca y nadie va a subir a reiniciarlo, así que hace falta un lenguaje que haga **exactamente** lo que se le escribe: nada de recolector de basura que se despierta cuando quiere, nada de magia escondida. C traduce casi uno a uno a lo que el procesador ejecuta, deja tocar el hardware directo y se comporta igual todas las veces. La contracara es que tampoco te cuida: si le pedís algo absurdo, lo hace con toda la cortesía del mundo.

La idea: todo lo que viene en este apunte apunta a lo mismo que la clase de Leandro pone en una diapositiva — escribir código **correcto, predecible y mantenible** para un sistema que no se puede reparar una vez lanzado —. Cada tema de C que aparezca va a tener esa pregunta atrás: ¿esto me ayuda a que el OBC no se cuelgue a 500 km de altura?

1.1 La plantilla de la cátedra

Todos los ejercicios de la materia arrancan de la misma plantilla, `template.c`. No es burocracia: ordena el archivo para que cualquiera —incluido vos, dentro de tres semanas— encuentre las cosas siempre en el mismo lugar. Ésta es la plantilla con lo mínimo adentro:

LA PLANTILLA, CON UN PROGRAMA ADENTRO — `m01-template.c`

```
/* C libraries */
#include <stdio.h>
#include <stdint.h>

/* Macros */
#define MISION "MISION-DEMO"

/* User libraries */

/* Global variables */

/* Functions declaration */

int main(void) {
    /* Main program */
    printf("OBC %s: arranque OK\n", MISION);
    return 0;
}

/* Functions definition */
```

SALIDA (copiada de la corrida real)
OBC MISION-DEMO: arranque OK

De arriba hacia abajo, cada sección tiene un trabajo:

Sección	Qué va
C libraries	Las bibliotecas del sistema, con <code>#include <...></code> . <code>stdio.h</code> trae <code>printf</code> ; <code>stdint.h</code> trae los tipos de ancho fijo del módulo que viene.

Macros	Las constantes con nombre, con <code>#define</code> . Acá, el nombre de la misión.
User libraries	Las bibliotecas propias, con <code>#include "..."</code> (comillas, no ángulos). Aparecen en el módulo 11.
Global variables	Lo que tiene que ver todo el archivo. Cuanto menos, mejor.
Functions declaration	La firma de cada función propia: nombre, qué recibe, qué devuelve. Va arriba de <code>main</code> para que el compilador la conozca antes de que alguien la llame.
main	Donde arranca el programa. Siempre <code>int main(void)</code> .
Functions definition	El cuerpo de cada función declarada arriba.

`int main(void)` se lee así: una función que se llama `main`, no recibe nada (`void`) y devuelve un entero (`int`). Ese entero es el `return 0` del final, y le avisa al sistema operativo que todo salió bien; cualquier otro número es un código de error. En la placa no hay sistema operativo que lo lea, pero en la PC sí, y la terminal lo usa.

Ojo: el punto y coma cierra cada **instrucción**, no cada renglón. Las llaves `{ }` agrupan instrucciones y no llevan punto y coma después. Olvidarse uno es el error número uno de toda la carrera, y el compilador lo va a reportar en la línea **siguiente**, porque recién ahí se da cuenta.

1.2 Compilar: de texto a programa

El procesador no entiende C: entiende instrucciones de máquina. El compilador traduce. En la terminal de Ubuntu, con el comando que pide la materia:

```
$ gcc -Wall -Wextra -std=c11 m01-template.c -o obc
$ ./obc
OBC MISION-DEMO: arranque OK
```

Pedazo	Qué hace
<code>gcc</code>	El compilador de C de GNU, el mismo que trae CubeIDE adentro (en su versión para ARM).
<code>-Wall</code>	Prende los avisos (<i>warnings</i>) más comunes. A pesar del nombre, no son todos .
<code>-Wextra</code>	Prende otro montón que <code>-Wall</code> deja afuera. Entre los dos, el compilador se vuelve un compañero desconfiado, que es lo que se quiere.
<code>-std=c11</code>	Compila según el estándar C de 2011. Así un programa se comporta igual en cualquier compilador que lo respete.
<code>-o obc</code>	El nombre del ejecutable que sale. Sin esto se llama <code>a.out</code> , que no le dice nada a nadie.
<code>./obc</code>	Lo corre. El <code>./</code> es «el que está en esta carpeta».

LO QUE PIDE LA CÁTEDRA — CERO WARNINGS

El práctico lo dice sin anestesia: un ejercicio **no está terminado** hasta que compila con **cero** warnings con esos tres flags. Compilar no alcanza; compilar callado, sí.

1.3 Leer un warning

Un warning es el compilador diciendo «esto es legal, pero me huele mal». El programa se genera igual, y por eso la tentación es ignorarlo. En software de vuelo un warning ignorado es una falla que todavía no ocurrió. Este programa declara una temperatura y nunca la usa:

UNA VARIABLE QUE NADIE USA — `m01-warning.c`

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    int16_t temperatura = -18; /* declarada... y olvidada */
    printf("Reporte enviado\n");
    return 0;
}
```

```
LO QUE DICE GCC (copiado de la corrida real)
m01-warning.c: In function 'main':
m01-warning.c:5:17: warning: unused variable 'temperatura' [-Wunused-variable]
   5 |         int16_t temperatura = -18; /* declarada... y olvidada */
     |         ^~~~~~
```

Se lee en tres partes, y siempre son las mismas. Primero **dónde**: `m01-warning.c:5:17` es archivo, línea 5, columna 17 (gcc cuenta el tabulador como ocho columnas). Después **qué**: `unused variable` (variable sin usar). Al final, entre corchetes, **qué flag lo prendió**: `-Wunused-variable`, que viene incluido en `-Wall`. Abajo, gcc dibuja el renglón y apunta con `^` al lugar exacto, que es más de lo que hacen muchos profesores.

Este warning en particular parece inofensivo, y casi siempre esconde algo peor: si declaraste la temperatura es porque la ibas a mandar en la telemetría, y te olvidaste. El compilador no sabe qué querías hacer, pero sabe que no lo hiciste.

LA POSTA

Un warning es un error que todavía no te mordió. Se leen de arriba hacia abajo, se arregla **el primero** y se vuelve a compilar: muchas veces los de abajo eran consecuencia de ése y desaparecen solos.

1.4 Un programa de vuelo no termina

En la PC, un programa arranca, hace lo suyo y termina. En el OBC no: si el programa terminara, el satélite se quedaría mudo hasta que alguien lo reinicie, y ese alguien está a varios cientos de kilómetros. Por eso todo programa embebido tiene la misma forma: **una inicialización que corre una vez**, y después **un bucle infinito** que repite lo mismo mientras haya energía —leer sensores, atender telecomandos, mandar telemetría—. Se llama *superloop*, y en la placa se escribe `while (1)`.

Para poder mostrarlo acá, el ejemplo lo corta a tres vueltas; en la placa esa cota no existe:

EL SUPERLOOP, RECORTADO PARA LA PC — `m01-superloop.c`

```
#include <stdio.h>
#include <stdint.h>

#define VUELTAS_DE_PRUEBA 3u /* en la PC lo cortamos; en la placa es while (1) */

int main(void) {
    uint32_t vuelta = 0;

    /* Inicializacion: corre UNA sola vez */
    printf("Iniciando perifericos...\n");

    /* Superloop: lo mismo, una y otra vez, mientras haya energia */
    while (vuelta < VUELTAS_DE_PRUEBA) {
        vuelta++;
        printf("Vuelta %u: leer sensores, atender telecomandos, mandar telemetria\n", vuelta);
    }
    return 0;
}
```

```
SALIDA (copiada de la corrida real)
Iniciando periféricos...
Vuelta 1: leer sensores, atender telecomandos, mandar telemetria
Vuelta 2: leer sensores, atender telecomandos, mandar telemetria
Vuelta 3: leer sensores, atender telecomandos, mandar telemetria
```

Ojo: que el bucle **principal** no termine no quiere decir que **cualquier** bucle pueda no terminar. Un **while** que espera a un sensor que se rompió cuelga al OBC entero. Los bucles de espera llevan siempre una cota: se ve en el módulo 5.

MEJORA, NO ES DE LA CÁTEDRA – COMPILAR DESDE VS CODE

El mismo comando se puede atar a una tecla en VS Code (una **tarea**, `tasks.json`), así no se tipea cada vez. La guía de IDEs de la materia explica cómo.

MÓDULO 2

Variables y tipos

AL TERMINAR ESTE MÓDULO VAS A PODER

Elegir el tipo de cada dato contestando dos preguntas —¿puede ser negativo? ¿cuánto rango necesita?—, saber cuánto ocupa en la PC y en la placa, imprimirlo con el especificador correcto, y reconocer las dos trampas que más telemetría arruinan: el desborde y la división entera.

2.1 Qué es una variable

Una variable es un pedazo de memoria con **nombre**. Tiene cuatro cosas, y conviene nombrarlas desde ya porque las cuatro vuelven: un **nombre** (*temperatura*), un **tipo** (*int16_t*: cuántos bits ocupa y cómo se leen), un **valor** (*-18*) y una **dirección**, el lugar de la memoria donde vive. La dirección la vas a ignorar hasta el módulo de punteros, y ese día va a ser la protagonista.

En C toda variable se **declara** antes de usarla, diciendo su tipo. Declarar e **inicializar** (darle valor) se puede hacer junto, y conviene:

```
int16_t temperatura = -18; /* declarada e inicializada */
uint16_t corriente;      /* declarada: vale... cualquier cosa */
```

Ojo: una variable local declarada sin valor **no vale cero**: vale lo que haya quedado en ese pedazo de memoria, que puede ser cualquier cosa y cambiar de una corrida a otra. Es el error que funciona en tu máquina y falla en la de otro. Se inicializa siempre, aunque sea en cero.

2.2 Los tipos de C, y por qué no alcanzan

C trae cuatro tipos básicos: *char* (un carácter, o un entero chico), *int* (entero), *float* (decimal de precisión simple) y *double* (decimal de precisión doble). Se les pueden agregar calificadores de tamaño (*short*, *long*) y de signo (*signed*, *unsigned*). Hasta acá, lo de cualquier curso de C.

El problema es que C **no garantiza** cuánto mide cada uno: lo decide cada compilador según la máquina. El estándar sólo promete mínimos —un *int* tiene **al menos** 16 bits—. Este programa pregunta los tamaños con *sizeof*, que devuelve cuántos bytes ocupa un tipo:

CUÁNTO MIDE CADA TIPO, EN LA PC —m02-sizeof.c

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    printf("char      : %zu byte\n", sizeof(char));
    printf("short     : %zu bytes\n", sizeof(short));
    printf("int        : %zu bytes\n", sizeof(int));
    printf("long       : %zu bytes\n", sizeof(long));
    printf("float      : %zu bytes\n", sizeof(float));
    printf("double     : %zu bytes\n", sizeof(double));
    printf("uint8_t    : %zu byte\n", sizeof(uint8_t));
    printf("int16_t    : %zu bytes\n", sizeof(int16_t));
    printf("uint32_t   : %zu bytes\n", sizeof(uint32_t));
    return 0;
}
```

```

SALIDA (copiada de la corrida real)
char      : 1 byte
short     : 2 bytes
int       : 4 bytes
long      : 8 bytes
float     : 4 bytes
double    : 8 bytes
uint8_t   : 1 byte
int16_t   : 2 bytes
uint32_t  : 4 bytes

```

Ésa es la salida en la PC con Ubuntu. El mismo programa, compilado con el gcc para ARM que trae CubeIDE, da **long** de 4 bytes, no de 8 (medido con el compilador de CubeIDE 1.18.1 para un Cortex-M4, el de la NUCLEO-F446RE). Mismo código, mismo nombre de tipo, la mitad de bits. Si ese **long** era un contador que viaja en un paquete de telemetría, la estación terrena lo va a leer mal y nadie va a entender por qué.

2.3 La solución: tipos de ancho fijo

La biblioteca `<stdint.h>` trae tipos cuyo nombre **dice** cuánto miden, en cualquier máquina:

Tipo	Bits	Rango
<code>uint8_t</code>	8	0 a 255
<code>int8_t</code>	8	-128 a 127
<code>uint16_t</code>	16	0 a 65 535
<code>int16_t</code>	16	-32 768 a 32 767
<code>uint32_t</code>	32	0 a 4 294 967 295
<code>int32_t</code>	32	-2 147 483 648 a 2 147 483 647

El nombre se lee solo: **u** es *unsigned* (sin signo), el número son los bits, y **_t** es «tipo». La regla del rango sale de contar: con n bits hay 2^n combinaciones. Sin signo van de 0 a $2^n - 1$; con signo, la mitad se usa para los negativos, y van de -2^{n-1} a $2^{n-1} - 1$. Por eso `int8_t` llega a 127 y no a 128: el cero ocupa un lugar del lado positivo.

LO QUE PIDE LA CÁTEDRA — ANCHO FIJO, SIEMPRE

La clase lo dice en mayúsculas: en código embebido se usan **siempre** tipos de ancho fijo, y **unsigned** por defecto para lo que son cantidades o patrones de bits. El tamaño tiene que ser inequívoco y explícito.

2.4 Con signo o sin signo: dos preguntas

Para cada dato, antes de escribir el tipo, se contestan dos preguntas: ¿**puede ser negativo?** y ¿**hasta dónde puede llegar?** Una rueda de reacción —la que gira para orientar el satélite— lo ilustra bien:

TELEMETRÍA DE UNA RUEDA DE REACCIÓN – m02-rueda.c

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint16_t muestra = 1207; /* contador de muestras: nunca es negativo */
    int16_t velocidad = -3450; /* rpm: el signo dice para que lado gira */
    float tension = 11.8f; /* V del bus de potencia */
    uint16_t corriente = 420; /* mA */

    /* mA -> A ANTES de multiplicar, o la potencia sale mil veces grande */
    float potencia = tension * (corriente / 1000.0f);

    printf("Muestra #%u\n", muestra);
    printf("Velocidad: %d rpm\n", velocidad);
    printf("Bus: %.1f V, %u mA -> %.2f W\n", tension, corriente, potencia);
    return 0;
}
```

SALIDA (copiada de la corrida real)

```
Muestra #1207
Velocidad: -3450 rpm
Bus: 11.8 V, 420 mA -> 4.96 W
```

El número de muestra no puede ser negativo y crece: `uint16_t`, que llega a 65 535. La velocidad sí puede ser negativa, porque la rueda gira para los dos lados y el signo es justamente **para cuál**: `int16_t`. La corriente en miliampere es una cantidad: `uint16_t`. La tensión tiene decimales: `float`.

Ojo: la corriente viene en **mili**ampere, y la potencia se calcula en watt. Si se multiplica sin convertir, el resultado sale mil veces más grande: el satélite estaría consumiendo 4956 W, más o menos lo de un horno eléctrico con el aire acondicionado prendido. Las unidades se convierten **antes** de operar, y el `1000.0f` (con punto) es a propósito: se ve en la última sección.

2.5 Imprimir con el especificador correcto

`printf` no sabe qué tipo le pasaste: se lo dice el especificador, y si no coinciden imprime cualquier cosa. Los que se usan en la materia:

Especificador	Para	Ejemplo
%d	enteros con signo	-3450
%u	enteros sin signo	1207
%f, %.2f	float y double; .2 son los decimales	4.96
%X, %02X	hexadecimal; 02 rellena con ceros a dos dígitos	0F
%c	un carácter	A
%s	una cadena	MISION-DEMO
%zu	lo que devuelve <code>sizeof</code>	4
%%	el signo de porcentaje mismo	%

Ojo: en la PC, `printf("%u", x)` con un `uint32_t` compila sin quejas. En la placa **no**: para el gcc de ARM, `uint32_t` es un `long unsigned int`, y -Wall avisa que %u espera un `unsigned int` (medido con el compilador de CubeIDE). El arreglo portable es la macro `PRIu32` de `<inttypes.h>`: `printf("%" PRIu32 "\n", x);`. Se ve fea, y funciona en las dos máquinas, que es lo que importa.

2.6 La primera trampa: el desborde

Un tipo tiene un rango, y C no avisa cuando se lo pasa. Un contador de paquetes de 8 bits:

UN CONTADOR DE 8 BITS QUE SE DA VUELTA — m02-desborde.c

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint8_t paquete = 254;    /* 8 bits: de 0 a 255, y ni uno mas */

    for (int i = 0; i < 3; i++) {
        printf("Paquete #%u\n", paquete);
        paquete++;            /* despues de 255 vuelve a 0, sin avisar */
    }
    return 0;
}
```

```
SALIDA (copiada de la corrida real)
Paquete #254
Paquete #255
Paquete #0
```

Después de 255 viene 0. En un tipo **sin signo** eso está definido por el estándar: la cuenta da la vuelta como un cuentakilómetros. Si la estación terrena usa ese número para detectar paquetes perdidos, va a creer que perdió 255 de golpe. El arreglo es elegir el ancho pensando en el **peor caso**: ¿cuántos paquetes manda el satélite en una pasada? ¿Y en un año?

Ojo: en un tipo **con signo** el desborde es todavía peor: el estándar dice que es *comportamiento indefinido*, o sea que el compilador puede hacer lo que quiera, incluido algo distinto según cuánto optimice. No es «da la vuelta a -128»: es «no se sabe». Con signo, se chequea **antes** de sumar.

2.7 La segunda trampa: la división entera

Si los dos operandos de una división son enteros, C hace división **entera**: tira los decimales, sin redondear. Calcular el porcentaje de batería parece una regla de tres, y es un campo minado:

TRES FORMAS DE CALCULAR UN PORCENTAJE — m02-division.c

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint16_t mv = 3790;    /* bateria: 3000 mV es 0 %, 4200 mV es 100 % */

    uint16_t mal = (mv - 3000) / (4200 - 3000) * 100;    /* divide primero: 790/1200 = 0 */
    uint16_t bien = (mv - 3000) * 100 / (4200 - 3000);    /* multiplica primero */
    float exact = (mv - 3000) * 100.0f / (4200 - 3000);    /* un float en la cuenta */

    printf("mal : %u %%\n", mal);
    printf("bien : %u %%\n", bien);
    printf("exact : %.1f %%\n", exact);
    return 0;
}
```

```
SALIDA (copiada de la corrida real)
mal : 0 %
bien : 65 %
exact : 65.8 %
```

La primera divide antes de multiplicar: $790 / 1200$ da 0 (es menos de uno, y la parte decimal se tira), y cero por cien es cero. La batería está al 65 % y el OBC cree que está vacía: según cómo esté programado, entra en modo seguro y apaga la carga útil. La segunda multiplica primero $79000 / 1200$ y da 65: los decimales se siguen tirando, pero al final, donde pesan menos. La tercera mete un **float** en la cuenta ($100.0f$) y C pasa **toda** la operación a punto flotante: 65,8.

LA POSTA

En C el tipo no es un detalle de escritura: decide cuánto entra, qué pasa cuando se pasa, y cómo se hace cada cuenta. Por cada variable, dos preguntas: ¿puede ser negativo? ¿hasta dónde llega? Y en cada división, una tercera: ¿estos dos son enteros?